

Etap 3: Implementacja aplikacji bazodanowej (5 godz.)

Cel etapu

Stworzenie kompletnego systemu, który integruje bazę danych z warstwą logiczną (np. w Pythonie). Na tym etapie "ożywiamy" projekt, implementując funkcjonalności dla użytkowników i administratorów.

Architektura aplikacji (Propozycja)

Zalecane jest podejście warstwowe, aby oddzielić zapytania SQL od logiki prezentacji:

1. **Warstwa Danych (SQL Scripts):** Pliki `.sql` tworzące strukturę i dane testowe.
2. **Warstwa Dostępu do Danych (DAO/Repository):** Funkcje w Pythonie wykonujące konkretne zapytania.
3. **Warstwa Logiki/Interfejsu:** Menu konsolowe lub proste GUI obsługujące interakcję z użytkownikiem.

Zadania na tym etapie

- ☐ **Inicjalizacja bazy (PostgreSQL):**
 - ☐ Stworzenie skryptu `schema.sql` (tabele, klucze, więzy, procedury z Etapu 2).
 - ☐ Użycie `GENERATED ALWAYS AS IDENTITY` dla kluczy głównych.
 - ☐ Przygotowanie raportu z atrybutami (zgodnie z wytycznymi - `information_schema.columns`).
- ☐ **Zasilenie bazy danymi (Seeding):**
 - ☐ Skrypt `seed.sql` dodający: min. 20 filmów, 10 użytkowników i 50 wypożyczeń.
 - ☐ Zadbanie o różnorodność danych (różne statusy, daty).
 - ☐ Przygotowanie prezentacji zawartości tabel (`SELECT * FROM ...`).
- ☐ **Implementacja Modułu Użytkownika:**
 - ☐ Rejestracja i logowanie (z walidacją danych).
 - ☐ Wyszukiwanie filmów (np. po tytule lub gatunku).
 - ☐ Proces wypożyczenia (obsługa transakcji).
- ☐ **Implementacja Modułu Administratora:**
 - ☐ Zarządzanie katalogiem (CRUD dla filmów).
 - ☐ Panel statystyk (podstawowe zliczenia).

Obsługa błędów bazy danych

Dobra aplikacja musi reagować na błędy zwracane przez PostgreSQL:

Wyjątek (psycopg)	Przyczyna	Akcja dla użytkownika
UniqueViolation	Próba rejestracji na zajęty e-mail.	"Ten adres e-mail jest już w użyciu."
ForeignKeyViolation	Próba usunięcia filmu, który jest wypożyczony.	"Nie można usunąć filmu z historią wypożyczeń."
CheckViolation	Cena filmu jest mniejsza niż 0.	"Wprowadź poprawną kwotę."
RaiseException	Wyzwalacz z Etapu 2 zablokował akcję.	Wyświetl komunikat z RAISE EXCEPTION .

Przykład struktury kodu (Python)

```
import psycopg
from contextlib import contextmanager

# 1. Zarządzanie połączeniem
@contextmanager
def get_conn():
    conn = psycopg.connect("dbname=vod user=postgres password=secret host=localhost")
    try:
        yield conn
    finally:
        conn.close()

# 2. Logika dostępu do danych (Repository)
class MovieRepository:
    def find_all(self, search_term=None):
        with get_conn() as conn, conn.cursor() as cur:
            sql = "SELECT tytuł, cena FROM film WHERE dostepny = TRUE"
            if search_term:
                sql += " AND tytuł ILIKE %s"
            cur.execute(sql, (f"%{search_term}%",))
        else:
            cur.execute(sql)
        return cur.fetchall()

    def rent_movie(self, user_id: int, movie_id: int):
        with get_conn() as conn:
            # Użycie transakcji gwarantuje spójność
            with conn.transaction():
                with conn.cursor() as cur:
                    cur.execute(
                        """
                        INSERT INTO wypozyczenie (uzytkownik_id, film_id, data_wypozyczenia)
                        VALUES (%s, %s, NOW())
                    """)
```

```
        """  
        (user_id, movie_id),  
    )
```

Wymagania techniczne (Checklist)

- ☐ Czy przygotowałeś tabele z atrybutami dla każdej z 5 encji?
- ☐ Czy przygotowałeś tabele z zawartością dla każdej z 5 encji?
- ☐ Czy wszystkie zapytania `INSERT/UPDATE` są wykonywane wewnątrz **transakcji**?
- ☐ Czy używasz **parametryzacji zapytań** (np. `%s` w `psycopg`) zamiast f-stringów?
- ☐ Czy kod jest podzielony na czytelne funkcje/klasy?
- ☐ Czy po uruchomieniu skryptu `schema.sql` baza jest gotowa do pracy?
- ☐ Czy skrypt `seed.sql` można uruchomić wielokrotnie bez błędów?
